Phase 3: React

컴포넌트 기반 UI 개발

Day 09 ~ Day 15

웹 풀스택 PBL 과정

Day 09 - React 시작: 컴포넌트와 JSX

React란 무엇인가?

React는 Facebook(현 Meta)에서 만든 **UI 라이브러리**입니다. 웹 페이지의 화면(UI)을 효율적으로 만들고 관리하기 위한 도구입니다.

실생활 비유: 레고 블록

React를 이해하는 가장 좋은 비유는 **레고 블록**입니다.

- **레고 블록 하나** = **컴포넌트(Component)** 하나
- 작은 블록(버튼, 입력창)을 조립해서 중간 블록(폼, 카드)을 만들고
- 중간 블록을 조립해서 큰 블록(페이지)을 만듭니다

```
전체 페이지 (App)
├── 헤더 (Header)
│   ├── 로고 (Logo)
│   └── 네비게이션 (Nav)
├── 본문 (Main)
│   ├── 검색바 (SearchBar)
│   └── 카드목록 (CardList)
│       ├── 카드 (Card)
│       ├── 카드 (Card)
│       └── 카드 (Card)
└── 푸터 (Footer)
```

왜 React가 필요한가?

Phase 1에서 DOM을 직접 조작했던 것을 기억하시나요?

```
// Phase 1 방식: DOM 직접 조작 (명령형)
const li = document.createElement('li');
li.textContent = '새 할일';
li.className = 'todo-item';
document.getElementById('todo-list').appendChild(li);
```

이 방식의 문제점:

- 코드가 길어지면 **어디서 무엇이 바뀌는지** 추적하기 어렵다
- UI가 복잡해지면 **버그가 급격히 증가**한다
- **재사용**이 어렵다 (같은 UI를 또 만들려면 코드를 복사해야 한다)

```
// React 방식: 선언형 (원하는 결과만 선언)
function TodoItem({ text }: { text: string }) {
  return <li className="todo-item">{text}</li>;
}
```

> **핵심 차이:** DOM 직접 조작은 "이렇게 해라"(명령형), React는 "이렇게 보여라"(선언형)입니다. 마치 요리를 할 때 "냄비에 물을 넣고, 불을 켜고, 면을 넣어라" vs "라면 한 그릇 주세요"의 차이와 같습니다.

Vite로 프로젝트 생성

create-react-app 대신 Vite를 쓰는 이유

비교 항목	create-react-app (CRA)	Vite
프로젝트 생성 속도	느림 (1~2분)	빠름 (몇 초)
개발 서버 시작	느림	매우 빠름
코드 변경 반영	느림	거의 즉시
유지보수 상태	사실상 중단	활발히 개발 중

프로젝트 생성 단계

```
# 1단계: Vite로 React + TypeScript 프로젝트 생성
npm create vite@latest my-first-react -- --template react-ts

# 2단계: 프로젝트 폴더로 이동
cd my-first-react

# 3단계: 의존성 설치
npm install

# 4단계: 개발 서버 시작
npm run dev
```

프로젝트 구조 이해

```
my-first-react/
├── node_modules/      # 설치된 라이브러리들 (건드리지 않음)
├── public/            # 정적 파일 (이미지 등)
├── src/               # 우리가 작업할 소스 코드
│   ├── App.tsx        # 메인 컴포넌트 (시작점)
│   ├── App.css        # App 컴포넌트의 스타일
│   ├── main.tsx       # React를 HTML에 연결하는 진입점
│   ├── index.css      # 전역 스타일
│   └── vite-env.d.ts  # Vite 타입 선언 (건드리지 않음)
├── index.html         # HTML 템플릿
├── package.json       # 프로젝트 설정
├── tsconfig.json      # TypeScript 설정
└── vite.config.ts     # Vite 설정
```

> **main.tsx가 하는 일:** HTML의 `<div id="root">` 안에 React 앱을 "심는" 역할을 합니다. 화분(root div)에 식물(React 앱)을 심는다고 생각하면 됩니다.

JSX란?

JSX는 **JavaScript 안에서 HTML처럼 쓸 수 있는 문법**입니다. "JavaScript XML"의 약자입니다.

```
// JSX: HTML처럼 보이지만, 사실은 JavaScript입니다!
const element = <h1>안녕하세요, React!</h1>;

// 위 코드는 실제로 이렇게 변환됩니다 (우리가 직접 쓸 필요 없음)
const element2 = React.createElement('h1', null, '안녕하세요, React!');
```

JSX 규칙 4가지

규칙 1: 하나의 루트 요소로 감싸야 한다

```
// 잘못된 예시: 두 개의 루트 요소
function Bad() {
  return (
    <h1>제목</h1>
    <p>내용</p>  // 에러 발생!
  );
}

// 올바른 예시 1: div로 감싸기
function Good1() {
  return (
    <div>
      <h1>제목</h1>
      <p>내용</p>
    </div>
  );
}

// 올바른 예시 2: Fragment로 감싸기 (불필요한 div 방지)
function Good2() {
  return (
    <>
      <h1>제목</h1>
      <p>내용</p>
    </>
  );
}
```

> **비유:** 택배를 보낼 때 여러 물건을 그냥 보낼 수 없고, 하나의 박스에 담아야 하는 것과 같습니다. `<>...</>` (Fragment)는 투명한 박스 같은 것입니다.

규칙 2: class 대신 className 사용

```
// HTML에서는 class
// <div class="container">내용</div>

// JSX에서는 className (class는 JavaScript 예약어이므로)
function MyComponent() {
  return <div className="container">내용</div>;
}
```

규칙 3: JavaScript 표현식은 중괄호 {} 안에

```
function Greeting() {
  const name = "김철수";          // 변수
  const age = 25;                 // 숫자
  const isAdult = age ≥ 18;       // boolean

  return (
    <div>
      {/* 변수 값 출력 */}
      <h1>안녕하세요, {name}님!</h1>

      {/* 계산식 사용 */}
      <p>내년에는 {age + 1}살이 됩니다.</p>

      {/* 함수 호출 */}
      <p>이름 길이: {name.length}글자</p>

      {/* 주의: if문은 직접 사용 불가! (표현식만 가능) */}
    </div>
  );
}
```

규칙 4: 모든 태그는 닫아야 한다

```
// HTML에서는 닫지 않아도 됨
// <img src="photo.jpg">
// <br>
// <input type="text">

// JSX에서는 반드시 닫아야 함 (self-closing)
function MyComponent() {
  return (
    <div>
      <img src="photo.jpg" />
      <br />
      <input type="text" />
    </div>
  );
}
```

조건부 렌더링

화면에 "조건에 따라 다르게 보여주기"를 조건부 렌더링이라고 합니다.

```
function UserStatus({ isLoggedIn }: { isLoggedIn: boolean }) {
  return (
    <div>
      {/* 방법 1: && 연산자 (조건이 true일 때만 보여주기) */}
      {isLoggedIn && <p>환영합니다!</p>}

      {/* 방법 2: 삼항연산자 (true일 때 / false일 때 다르게) */}
      {isLoggedIn ? (
        <button>로그아웃</button>
      ) : (
        <button>로그인</button>
      )}
    </div>
  );
}
```

> **&& 연산자 주의:** `{0 && <p>보임</p>}` 은 화면에 `0` 이 출력됩니다! 숫자 0은 falsy이지만 React가 화면에 렌더링합니다. `{count > 0 && <p>{count}개</p>}` 처럼 boolean으로 변환해서 사용하세요.

컴포넌트 (함수 컴포넌트)

컴포넌트는 **UI 조각을 반환하는 함수**입니다.

```
// 가장 간단한 컴포넌트
function Welcome() {
  return <h1>환영합니다!</h1>;
}

// 컴포넌트 사용 (HTML 태그처럼 사용)
function App() {
  return (
    <div>
      <Welcome />
      <Welcome />
      <Welcome />
    </div>
  );
}
```

네이밍 규칙

```
// 올바른 네이밍: PascalCase (첫 글자 대문자)
function TodoItem() { return <li>할일</li>; }
function UserProfile() { return <div>프로필</div>; }
function SearchBar() { return <input />; }

// 잘못된 네이밍: 소문자로 시작하면 HTML 태그로 인식!
// function todoItem() { ... }  // React가 컴포넌트로 인식하지 못함
```

Props: 부모에서 자식으로 데이터 전달

비유: 택배 배달

Props는 **부모 컴포넌트가 자식 컴포넌트에게 보내는 택배**입니다.

- 부모가 택배(데이터)를 보냄
- 자식은 받아서 사용만 함 (수정 불가!)
- 택배 내용물(props)은 **읽기 전용**

```typescript
// TypeScript로 Props 타입 정의
interface GreetingProps {
  name: string;     // 이름 (필수)
  age: number;      // 나이 (필수)
  hobby?: string;   // 취미 (선택 - ?가 붙으면 없어도 됨)
}

// 자식 컴포넌트: props를 받아서 사용
function Greeting({ name, age, hobby }: GreetingProps) {
  return (
    <div>
      <h2>{name}님, 안녕하세요!</h2>
      <p>나이: {age}살</p>
      {/* hobby가 있을 때만 표시 */}
      {hobby && <p>취미: {hobby}</p>}
    </div>
  );
}

// 부모 컴포넌트: props를 전달
function App() {
  return (
    <div>
      {/* 속성(attribute)처럼 값을 전달 */}
      <Greeting name="김철수" age={25} hobby="축구" />
      <Greeting name="이영희" age={30} />
    </div>
  );
}
```

children Props

`children`은 **컴포넌트 태그 사이에 들어가는 내용**입니다.

```
// children을 받는 컴포넌트
interface CardProps {
  title: string;
  children: React.ReactNode;  // 어떤 JSX든 받을 수 있음
}

function Card({ title, children }: CardProps) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <div className="card-body">
        {children}  {/* 부모가 넣어준 내용이 여기에 표시됨 */}
      </div>
    </div>
  );
}

// 사용 예시
function App() {
  return (
    <Card title="공지사항">
      {/* 이 사이의 모든 내용이 children */}
      <p>내일은 휴일입니다.</p>
      <p>즐거운 하루 되세요!</p>
    </Card>
  );
}
```

> **비유:** children은 **선물 포장**과 같습니다. Card 컴포넌트는 포장지(테두리, 제목)이고, children은 그 안에 넣는 선물(내용)입니다. 같은 포장지에 다른 선물을 넣을 수 있습니다.

리스트 렌더링: map + key

배열 데이터를 화면에 보여줄 때 `map()` 을 사용합니다.

```typescript
interface Todo {
  id: number;
  text: string;
  done: boolean;
}

function TodoList() {
  // 할일 목록 데이터
  const todos: Todo[] = [
    { id: 1, text: "장보기", done: false },
    { id: 2, text: "청소하기", done: true },
    { id: 3, text: "공부하기", done: false },
  ];

  return (
    <ul>
      {/* map으로 배열을 JSX 배열로 변환 */}
      {todos.map((todo) => (
        <li key={todo.id}>
          {/* done이면 취소선 스타일 적용 */}
          <span style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>
            {todo.text}
          </span>
        </li>
      ))}
    </ul>
  );
}
```

key가 왜 필요한가?

> **비유: 학교 출석부의 학생 번호**
>
> 반에 학생이 30명 있습니다. 선생님이 누가 전학 갔고 누가 새로 왔는지 확인하려면 **학생 번호**(key)가 있어야 빠르게 찾을 수 있습니다. 번호 없이 이름만으로 찾으면 한 명 한 명 확인해야 합니다.

React도 마찬가지입니다. key가 있으면 어떤 항목이 추가/삭제/변경되었는지 **빠르게 파악**할 수 있습니다.

```
// 좋은 key: 고유한 id 사용
{todos.map((todo) => (
  <li key={todo.id}>{todo.text}</li>  // id는 각 항목마다 고유함
))}

// 나쁜 key: index 사용 (순서가 바뀌면 문제 발생!)
{todos.map((todo, index) => (
  <li key={index}>{todo.text}</li>  // 항목이 추가/삭제되면 index가 바뀜
))}
```

자주 하는 실수

```
// 실수 1: key를 빼먹음
// Warning: Each child in a list should have a unique "key" prop.
{todos.map((todo) => (
  <li>{todo.text}</li>  // key가 없음!
))}

// 실수 2: map 안에서 return을 빼먹음
{todos.map((todo) => {
  <li key={todo.id}>{todo.text}</li>  // return이 없음!
})}

// 올바른 방법 (중괄호면 return 필요, 소괄호면 자동 return)
{todos.map((todo) => (
  <li key={todo.id}>{todo.text}</li>  // 소괄호 () 사용 = 자동 return
))}
```

Day 10 - 컴포넌트 분리와 합성

컴포넌트를 왜 나누는가?

비유: 식당 주방의 분업

큰 식당의 주방을 생각해보세요:

- **전채 담당 셰프** = Header 컴포넌트
- **메인 요리 담당 셰프** = Main 컴포넌트
- **디저트 담당 셰프** = Footer 컴포넌트

한 명이 모든 요리를 하면:

- 실수가 많아짐 (모든 코드가 한 파일에)
- 수정하기 어려움 (어디를 고쳐야 할지 모름)
- 협업 불가능 (두 사람이 같은 파일을 동시에 수정하기 어려움)

분업하면:

- 각자 맡은 부분만 집중 (관심사 분리)
- 문제가 생기면 담당 셰프만 확인 (디버깅 용이)
- 다른 식당에서도 같은 셰프를 부를 수 있음 (재사용성)

컴포넌트 분리 기준

하나의 컴포넌트는 하나의 역할만 해야 합니다.

Before: 모든 것이 하나의 컴포넌트에

```
// 나쁜 예: App.tsx에 모든 코드가 들어있음
function App() {
  const todos = [
    { id: 1, text: "장보기", done: false },
    { id: 2, text: "청소하기", done: true },
  ];

  return (
    <div>
      {/* 헤더 영역 */}
      <header>
        <h1>나의 할일 목록</h1>
        <p>총 {todos.length}개의 할일</p>
      </header>

      {/* 입력 영역 */}
      <form>
        <input type="text" placeholder="할일을 입력하세요" />
        <button type="submit">추가</button>
      </form>

      {/* 목록 영역 */}
      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>
            <input type="checkbox" checked={todo.done} />
            <span>{todo.text}</span>
            <button>삭제</button>
          </li>
        ))}
      </ul>

      {/* 푸터 영역 */}
      <footer>
        <p>완료: {todos.filter(t => t.done).length}개</p>
      </footer>
    </div>
  );
}
```

After: 역할별로 컴포넌트 분리

```tsx
// --- components/TodoHeader.tsx ---
interface TodoHeaderProps {
  totalCount: number;  // 전체 할일 개수
}

function TodoHeader({ totalCount }: TodoHeaderProps) {
  return (
    <header>
      <h1>나의 할일 목록</h1>
      <p>총 {totalCount}개의 할일</p>
    </header>
  );
}

export default TodoHeader;
```

```tsx
// --- components/TodoForm.tsx ---
interface TodoFormProps {
  onAdd: (text: string) => void;  // 할일 추가 콜백 함수
}

function TodoForm({ onAdd }: TodoFormProps) {
  const [text, setText] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();           // 폼 기본 동작(페이지 새로고침) 방지
    if (text.trim() === "") return; // 빈 문자열 방지
    onAdd(text);                  // 부모에게 새 할일 텍스트 전달
    setText("");                  // 입력창 초기화
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="할일을 입력하세요"
      />
      <button type="submit">추가</button>
    </form>
  );
}

export default TodoForm;
```

```typescript
// --- components/TodoItem.tsx ---
interface TodoItemProps {
  id: number;
  text: string;
  done: boolean;
  onToggle: (id: number) => void;    // 완료 토글 콜백
  onDelete: (id: number) => void;    // 삭제 콜백
}

function TodoItem({ id, text, done, onToggle, onDelete }: TodoItemProps) {
  return (
    <li>
      <input
        type="checkbox"
        checked={done}
        onChange={() => onToggle(id)}  // 체크박스 클릭 시 토글
      />
      <span style={{ textDecoration: done ? 'line-through' : 'none' }}>
        {text}
      </span>
      <button onClick={() => onDelete(id)}>삭제</button>
    </li>
  );
}

export default TodoItem;
```

```react
// --- App.tsx (깔끔해진 메인 컴포넌트) ---
import TodoHeader from './components/TodoHeader';
import TodoForm from './components/TodoForm';
import TodoItem from './components/TodoItem';

function App() {
  const [todos, setTodos] = useState([
    { id: 1, text: "장보기", done: false },
    { id: 2, text: "청소하기", done: true },
  ]);

  return (
    <div>
      <TodoHeader totalCount={todos.length} />
      <TodoForm onAdd={(text) => { /* 추가 로직 */ }} />
      <ul>
        {todos.map((todo) => (
          <TodoItem
            key={todo.id}
            {...todo}
            onToggle={(id) => { /* 토글 로직 */ }}
            onDelete={(id) => { /* 삭제 로직 */ }}
          />
        ))}
      </ul>
    </div>
  );
}
```

콜백 Props: 자식에서 부모로 통신

비유: 호텔 프론트 데스크에 전화하기

- **부모 컴포넌트** = 호텔 프론트 데스크
- **자식 컴포넌트** = 호텔 객실의 손님
- **콜백 함수** = 프론트 데스크의 전화번호

손님(자식)이 직접 호텔 시스템을 바꿀 수 없습니다. 대신 프론트(부모)에 전화(콜백 호출)해서 요청합니다.

```
// 부모: 전화번호(콜백 함수)를 자식에게 전달
function Parent() {
  const [items, setItems] = useState<string[]>([]);

  // 이 함수를 자식에게 "전화번호"로 전달
  const handleDelete = (index: number) => {
    setItems(prev => prev.filter((_, i) => i !== index));
  };

  return (
    <div>
      {items.map((item, index) => (
        <Child
          key={index}
          text={item}
          onDelete={() => handleDelete(index)}  // 전화번호 전달
        />
      ))}
    </div>
  );
}

// 자식: 필요할 때 "전화"를 건다 (콜백 호출)
function Child({ text, onDelete }: { text: string; onDelete: () => void }) {
  return (
    <div>
      <span>{text}</span>
      <button onClick={onDelete}>삭제</button>  {/* 전화 걸기! */}
    </div>
  );
}
```

onDelete, onToggle 패턴

React에서 콜백 props의 이름은 보통 `on` + 동사 형태로 짓습니다:

콜백 이름	의미	사용 예시
`onDelete`	삭제 요청	삭제 버튼 클릭
`onToggle`	토글 요청	체크박스 클릭
`onEdit`	수정 요청	수정 버튼 클릭
`onSubmit`	제출 요청	폼 제출
`onChange`	변경 알림	입력값 변경

조건부 렌더링 심화

실제 앱에서는 여러 상태에 따라 다른 UI를 보여줘야 합니다.

```
interface TodoListProps {
  todos: Todo[];
  isLoading: boolean;
  error: string | null;
}

function TodoList({ todos, isLoading, error }: TodoListProps) {
  // 상태 1: 로딩 중
  if (isLoading) {
    return <div className="loading">불러오는 중...</div>;
  }

  // 상태 2: 에러 발생
  if (error) {
    return <div className="error">오류: {error}</div>;
  }

  // 상태 3: 데이터가 비어있음
  if (todos.length === 0) {
    return <div className="empty">할일이 없습니다. 새로 추가해보세요!</div>;
  }

  // 상태 4: 정상적으로 데이터 표시
  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem key={todo.id} {...todo} />
      ))}
    </ul>
  );
}
```

> **팁:** 함수 상단에서 "예외 상태"를 먼저 처리하고 return하면 (Early Return 패턴), 코드가 깔끔해집니다.

컴포넌트 합성 (Composition)

children을 활용한 Layout 패턴

```
// 재사용 가능한 레이아웃 컴포넌트
interface PageLayoutProps {
  title: string;
  children: React.ReactNode;
}

function PageLayout({ title, children }: PageLayoutProps) {
  return (
    <div className="page">
      <header className="page-header">
        <h1>{title}</h1>
      </header>
      <main className="page-content">
        {children}  {/* 각 페이지마다 다른 내용이 여기에 들어감 */}
      </main>
      <footer className="page-footer">
        <p>Copyright 2026</p>
      </footer>
    </div>
  );
}

// 사용: 같은 레이아웃, 다른 내용
function HomePage() {
  return (
    <PageLayout title="홈">
      <p>환영합니다!</p>
    </PageLayout>
  );
}

function AboutPage() {
  return (
    <PageLayout title="소개">
      <p>이 사이트는...</p>
    </PageLayout>
  );
}
```

자주 하는 실수

```
// 실수: 콜백 함수를 즉시 실행해버림
<button onClick={handleDelete(id)}>삭제</button>
// handleDelete(id)는 "즉시 실행"됨! 렌더링할 때 바로 삭제됨!

// 올바른 방법 1: 화살표 함수로 감싸기
<button onClick={() => handleDelete(id)}>삭제</button>

// 올바른 방법 2: 인자가 없으면 함수 참조만 전달
<button onClick={handleDelete}>삭제</button>
```

Day 11 - useState로 상태 관리 (CRUD)

상태(State)란?

비유: 사람의 기분

상태(State)는 **시간에 따라 변할 수 있는 데이터**입니다.

- 사람의 **기분**: 기쁨 → 슬픔 → 화남 (시간에 따라 변함)
- Todo 앱의 **할일 목록**: 추가/삭제/수정으로 변함
- 로그인 **상태**: 로그인됨 ↔ 로그아웃됨

> **Props vs State:**
>
> - Props = 부모에게서 받은 **용돈** (내가 바꿀 수 없음)
> - State = 내 **저금통** (내가 직접 넣고 뺄 수 있음)

---

useState 기본

```
import { useState } from 'react';

function Counter() {
  // useState(초기값) → [현재값, 변경함수] 반환
  // count: 현재 값 (처음에는 0)
  // setCount: 값을 변경하는 함수
  const [count, setCount] = useState<number>(0);

  return (
    <div>
      <p>카운트: {count}</p>
      {/* setCount로 값을 변경하면 화면이 자동으로 업데이트됨! */}
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
      <button onClick={() => setCount(0)}>초기화</button>
    </div>
  );
}
```

왜 직접 수정하면 안 되는가? (불변성!)

```
function WrongWay() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    // 절대 이렇게 하면 안 됩니다!
    // count = count + 1;  // 에러! const는 재할당 불가
    // 설령 let으로 바꿔도 React가 변경을 감지하지 못함

    // 반드시 setCount를 사용해야 React가 "아, 값이 바뀌었구나" 알고
    // 화면을 다시 그립니다 (리렌더링)
    setCount(count + 1);
  };

  return <button onClick={handleClick}>{count}</button>;
}
```

> **비유:** React는 **우체부**와 같습니다. 우체부(React)에게 "주소가 바뀌었어요"(setState)라고 알려줘야 새 주소로 배달합니다. 몰래 이사하면(직접 수정) 우체부는 모릅니다.

CRUD 패턴: 배열/객체 불변 업데이트

CRUD는 Create(추가), Read(조회), Update(수정), Delete(삭제)의 약자입니다.

전체 Todo CRUD 예제

```typescript
import { useState } from 'react';

// 타입 정의
interface Todo {
  id: number;
  text: string;
  done: boolean;
}

function TodoApp() {
  // 할일 목록 상태
  const [todos, setTodos] = useState<Todo[]>([
    { id: 1, text: "React 공부하기", done: false },
    { id: 2, text: "점심 먹기", done: true },
  ]);

  // 입력값 상태
  const [input, setInput] = useState("");

  // 다음 ID (간단한 방법)
  const [nextId, setNextId] = useState(3);

  // ═══ Create (추가) ═══
  // 기존 배열을 복사하고 새 항목을 뒤에 붙인다
  // [...기존배열, 새항목]
  const handleAdd = () => {
    if (input.trim() === "") return;

    const newTodo: Todo = {
      id: nextId,
      text: input,
      done: false,
    };

    setTodos([...todos, newTodo]);  // 기존 배열 + 새 항목
    setNextId(nextId + 1);          // ID 증가
    setInput("");                   // 입력창 초기화
  };

  // ═══ Delete (삭제) ═══
  // filter로 해당 id가 아닌 것들만 남긴다
  // prev.filter(item => item.id !== 삭제할id)
  const handleDelete = (id: number) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  // ═══ Update (수정 - 완료 토글) ═══
  // map으로 해당 id의 항목만 변경하고 나머지는 그대로
  // prev.map(item => item.id === id ? {...item, 변경사항} : item)
  const handleToggle = (id: number) => {
    setTodos(
      todos.map(todo =>
```

```
      todo.id === id
        ? { ...todo, done: !todo.done }  // 해당 항목만 done을 반전
        : todo                           // 나머지는 그대로
    )
  );
};

// === Update (수정 - 텍스트 변경) ===
const handleEdit = (id: number, newText: string) => {
  setTodos(
    todos.map(todo =>
      todo.id === id
        ? { ...todo, text: newText }  // 해당 항목의 text만 변경
        : todo                        // 나머지는 그대로
    )
  );
};

return (
  <div>
    <h1>할일 목록</h1>

    {/* 입력 폼 */}
    <div>
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && handleAdd()}
        placeholder="할일을 입력하세요"
      />
      <button onClick={handleAdd}>추가</button>
    </div>

    {/* 할일 목록 (Read) */}
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.done}
            onChange={() => handleToggle(todo.id)}
          />
          <span style={{
            textDecoration: todo.done ? 'line-through' : 'none'
          }}>
            {todo.text}
          </span>
          <button onClick={() => handleDelete(todo.id)}>삭제</button>
        </li>
      ))}
    </ul>
  </div>
```

```
  );
}
```

CRUD 패턴 정리 (그림으로 이해하기)

```
=== 추가 (Create) ===
기존: [A, B, C]
추가: [...기존, D]
결과: [A, B, C, D]    ← 새 배열이 만들어짐 (기존 배열은 그대로)

=== 삭제 (Delete) - B를 삭제 ===
기존: [A, B, C]
삭제: 기존.filter(x ⇒ x ≠ B)
결과: [A, C]          ← B만 빠진 새 배열

=== 수정 (Update) - B를 B'으로 변경 ===
기존: [A, B, C]
수정: 기존.map(x ⇒ x ═ B ? B' : x)
결과: [A, B', C]      ← B만 바뀐 새 배열
```

> **핵심:** 모든 경우에 **새로운 배열**을 만듭니다. 원본 배열을 직접 수정(push, splice 등)하면 안 됩니다!

이벤트 핸들링

```
function EventExamples() {
  // onClick: 클릭 이벤트
  const handleClick = () => {
    alert("버튼이 클릭되었습니다!");
  };

  // onChange: 입력값 변경 이벤트
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    console.log("입력값:", e.target.value);
  };

  // onSubmit: 폼 제출 이벤트
  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();  // 폼 제출 시 페이지 새로고침 방지!
    console.log("폼이 제출되었습니다!");
  };

  return (
    <div>
      <button onClick={handleClick}>클릭</button>

      <input onChange={handleChange} />

      <form onSubmit={handleSubmit}>
        <input type="text" />
        <button type="submit">제출</button>
      </form>
    </div>
  );
}
```

> **e.preventDefault()를 빼먹으면?** 폼이 제출될 때 브라우저가 페이지를 새로고침합니다. React 앱에서는 이것이 모든 상태를 초기화시키므로, 반드시 막아야 합니다.

파생 상태: 계산된 값은 state로 만들지 않기

```
function TodoStats() {
  const [todos, setTodos] = useState<Todo[]>([
    { id: 1, text: "공부", done: true },
    { id: 2, text: "운동", done: false },
    { id: 3, text: "청소", done: true },
  ]);

  // 나쁜 예: 완료 개수를 별도의 state로 관리
  // const [doneCount, setDoneCount] = useState(2);
  // → todos가 바뀔 때마다 doneCount도 같이 바꿔야 함 (동기화 문제!)

  // 좋은 예: todos로부터 계산 (파생 상태)
  const totalCount = todos.length;                          // 전체 개수
  const doneCount = todos.filter(t => t.done).length;       // 완료 개수
  const remainCount = totalCount - doneCount;               // 미완료 개수
  const progress = totalCount > 0
    ? Math.round((doneCount / totalCount) * 100)
    : 0;                                                    // 진행률(%)

  return (
    <div>
      <p>전체: {totalCount}개</p>
      <p>완료: {doneCount}개</p>
      <p>남은: {remainCount}개</p>
      <p>진행률: {progress}%</p>
    </div>
  );
}
```

> **원칙:** 다른 state로부터 계산할 수 있는 값은 state로 만들지 마세요. 그냥 변수로 계산하면 됩니다. state가 많아지면 동기화 버그가 발생합니다.

장바구니 예제 (복잡한 상태)

```typescript
interface CartItem {
  id: number;
  name: string;
  price: number;
  quantity: number;
}

function ShoppingCart() {
  const [cart, setCart] = useState<CartItem[]>([]);

  // 상품 추가 (이미 있으면 수량 증가)
  const addToCart = (product: { id: number; name: string; price: number }) => {
    const existing = cart.find(item => item.id === product.id);

    if (existing) {
      // 이미 있으면: 해당 상품의 수량만 +1
      setCart(
        cart.map(item =>
          item.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        )
      );
    } else {
      // 없으면: 새로 추가 (수량 1)
      setCart([...cart, { ...product, quantity: 1 }]);
    }
  };

  // 수량 변경
  const updateQuantity = (id: number, newQuantity: number) => {
    if (newQuantity <= 0) {
      // 수량이 0 이하면 삭제
      setCart(cart.filter(item => item.id !== id));
    } else {
      setCart(
        cart.map(item =>
          item.id === id ? { ...item, quantity: newQuantity } : item
        )
      );
    }
  };

  // 파생 상태: 총 금액 계산
  const totalPrice = cart.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  // 파생 상태: 총 수량 계산
  const totalQuantity = cart.reduce(
    (sum, item) => sum + item.quantity,
```

```
    0
  );

  return (
    <div>
      <h2>장바구니 ({totalQuantity}개)</h2>
      {cart.length === 0 ? (
        <p>장바구니가 비어있습니다.</p>
      ) : (
        <>
          {cart.map(item => (
            <div key={item.id}>
              <span>{item.name}</span>
              <span>{item.price.toLocaleString()}원</span>
              <button onClick={() => updateQuantity(item.id, item.quantity - 1)}>
              <span>{item.quantity}</span>
              <button onClick={() => updateQuantity(item.id, item.quantity + 1)}>
              <span>소계: {(item.price * item.quantity).toLocaleString()}원</span>
            </div>
          ))}
          <hr />
          <p>총 금액: {totalPrice.toLocaleString()}원</p>
        </>
      )}
    </div>
  );
}
```

Day 12 - 필터링, 검색, 정렬, useMemo

필터 + 검색 + 정렬 조합

실제 앱에서는 데이터를 필터링하고, 검색하고, 정렬해서 보여줘야 합니다. 단계적으로 구현해봅시다.

1단계: 필터만 구현

```typescript
import { useState } from 'react';

interface Product {
  id: number;
  name: string;
  category: string;
  price: number;
  inStock: boolean;
}

// 샘플 데이터
const PRODUCTS: Product[] = [
  { id: 1, name: "노트북", category: "전자기기", price: 1200000, inStock: true },
  { id: 2, name: "마우스", category: "전자기기", price: 35000, inStock: true },
  { id: 3, name: "책상", category: "가구", price: 250000, inStock: false },
  { id: 4, name: "의자", category: "가구", price: 180000, inStock: true },
  { id: 5, name: "모니터", category: "전자기기", price: 450000, inStock: false },
  { id: 6, name: "램프", category: "가구", price: 45000, inStock: true },
];

function ProductFilter() {
  // 필터 상태: 어떤 카테고리를 보여줄 것인가
  const [category, setCategory] = useState<string>("전체");

  // 카테고리 목록 추출 (중복 제거)
  const categories = ["전체", ...new Set(PRODUCTS.map(p => p.category))];

  // 필터링된 상품 목록 (파생 상태)
  const filteredProducts = category === "전체"
    ? PRODUCTS
    : PRODUCTS.filter(p => p.category === category);

  return (
    <div>
      {/* 카테고리 필터 버튼 */}
      <div>
        {categories.map(cat => (
          <button
            key={cat}
            onClick={() => setCategory(cat)}
            style={{
              fontWeight: category === cat ? 'bold' : 'normal',
              backgroundColor: category === cat ? '#4299e1' : '#e2e8f0',
              color: category === cat ? 'white' : 'black',
            }}
          >
            {cat}
          </button>
        ))}
      </div>

      {/* 상품 목록 */}
```

```
      <p>{filteredProducts.length}개의 상품</p>
      <ul>
        {filteredProducts.map(product => (
          <li key={product.id}>
            {product.name} - {product.price.toLocaleString()}원
            {!product.inStock && " (품절)"}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

2단계: 검색 추가

```
function ProductFilterAndSearch() {
  const [category, setCategory] = useState<string>("전체");
  const [search, setSearch] = useState<string>("");  // 검색어 상태 추가

  const categories = ["전체", ...new Set(PRODUCTS.map(p => p.category))];

  // 필터링 + 검색 적용 (체이닝)
  const filteredProducts = PRODUCTS
    // 1단계: 카테고리 필터
    .filter(p => category === "전체" || p.category === category)
    // 2단계: 검색어 필터 (이름에 검색어가 포함되어 있는지)
    .filter(p => p.name.toLowerCase().includes(search.toLowerCase()));

  return (
    <div>
      {/* 검색 입력 */}
      <input
        type="text"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        placeholder="상품명 검색..."
      />

      {/* 카테고리 필터 버튼 */}
      <div>
        {categories.map(cat => (
          <button key={cat} onClick={() => setCategory(cat)}>
            {cat}
          </button>
        ))}
      </div>

      {/* 결과 */}
      <p>{filteredProducts.length}개의 상품</p>
      <ul>
        {filteredProducts.map(product => (
          <li key={product.id}>
            {product.name} - {product.price.toLocaleString()}원
          </li>
        ))}
      </ul>
    </div>
  );
}
```

3단계: 정렬 추가

```
// 정렬 옵션 타입
type SortKey = "name" | "price";
type SortOrder = "asc" | "desc";

function ProductFilterSearchSort() {
  const [category, setCategory] = useState<string>("전체");
  const [search, setSearch] = useState<string>("");
  const [sortKey, setSortKey] = useState<SortKey>("name");     // 정렬 기준
  const [sortOrder, setSortOrder] = useState<SortOrder>("asc"); // 정렬 방향

  const categories = ["전체", ...new Set(PRODUCTS.map(p => p.category))];

  // 필터 → 검색 → 정렬 (순서대로 적용)
  const result = PRODUCTS
    .filter(p => category === "전체" || p.category === category)
    .filter(p => p.name.toLowerCase().includes(search.toLowerCase()))
    .sort((a, b) => {
      // 정렬 기준에 따라 비교
      let comparison = 0;
      if (sortKey === "name") {
        comparison = a.name.localeCompare(b.name);  // 문자열 비교 (한글 지원)
      } else if (sortKey === "price") {
        comparison = a.price - b.price;              // 숫자 비교
      }

      // 내림차순이면 부호 반전
      return sortOrder === "asc" ? comparison : -comparison;
    });

  // 정렬 토글 함수
  const handleSort = (key: SortKey) => {
    if (sortKey === key) {
      // 같은 컬럼을 다시 클릭하면 방향 전환
      setSortOrder(prev => prev === "asc" ? "desc" : "asc");
    } else {
      // 다른 컬럼을 클릭하면 그 컬럼으로 오름차순 정렬
      setSortKey(key);
      setSortOrder("asc");
    }
  };

  return (
    <div>
      <input
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        placeholder="상품명 검색..."
      />

      <div>
        {categories.map(cat => (
          <button key={cat} onClick={() => setCategory(cat)}>{cat}</button>
```

```
        ))}
      </div>

      <table>
        <thead>
          <tr>
            {/* 클릭 가능한 정렬 헤더 */}
            <th onClick={() => handleSort("name")} style={{ cursor: 'pointer' }}>
              상품명 {sortKey === "name" && (sortOrder === "asc" ? " ▲" : " ▼")}
            </th>
            <th onClick={() => handleSort("price")} style={{ cursor: 'pointer' }}>
              가격 {sortKey === "price" && (sortOrder === "asc" ? " ▲" : " ▼")}
            </th>
            <th>카테고리</th>
            <th>재고</th>
          </tr>
        </thead>
        <tbody>
          {result.map(product => (
            <tr key={product.id}>
              <td>{product.name}</td>
              <td>{product.price.toLocaleString()}원</td>
              <td>{product.category}</td>
              <td>{product.inStock ? "있음" : "품절"}</td>
            </tr>
          ))}
        </tbody>
      </table>

      <p>총 {result.length}개의 상품</p>
    </div>
  );
}
```

---

useMemo: 비용이 큰 계산 캐시하기

비유: 계산기의 M+ 버튼

계산기에서 복잡한 계산을 했을 때, 결과를 **M+(메모리 저장)** 버튼으로 저장해두면 나중에 다시 계산하지 않아도 됩니다. `useMemo`가 바로 이 M+ 버튼입니다.

```
import { useState, useMemo } from 'react';

function ExpensiveCalculation() {
  const [todos, setTodos] = useState<Todo[]>(/* 1000개의 할일 */);
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

  // useMemo 없이: theme이 바뀔 때마다 이 계산이 다시 실행됨!
  // const stats = {
  //   total: todos.length,
  //   done: todos.filter(t => t.done).length,
  //   categories: [...new Set(todos.map(t => t.category))],
  // };

  // useMemo 사용: todos가 바뀔 때만 다시 계산
  const stats = useMemo(() => {
    console.log("통계 계산 중...");  // todos가 바뀔 때만 이 로그가 출력됨
    return {
      total: todos.length,
      done: todos.filter(t => t.done).length,
      categories: [...new Set(todos.map(t => t.category))],
    };
  }, [todos]);  // 의존성 배열: todos가 바뀔 때만 다시 계산

  return (
    <div className={theme}>
      {/* theme이 바뀌어도 stats는 다시 계산되지 않음! */}
      <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
        테마 전환
      </button>
      <p>전체: {stats.total}, 완료: {stats.done}</p>
    </div>
  );
}
```

언제 useMemo를 쓰고, 언제 안 쓰는가?

상황	useMemo 필요?	이유
단순한 필터링 (10~20개)	X	계산이 빠르므로 캐시 오버헤드가 더 큼
대량 데이터 필터/정렬 (1000개+)	O	매 렌더링마다 재계산하면 느려짐
문자열 합치기	X	너무 간단한 연산
복잡한 통계/집계 계산	O	비용이 큰 계산

> **원칙:** 성능 문제가 **실제로 느껴질 때** 사용하세요. 처음부터 모든 곳에 useMemo를 쓰면 오히려 코드가 복잡해집니다. "너무 이른 최적화는 만악의 근원"이라는 프로그래밍 격언이 있습니다.

정렬 가능한 테이블 (완성 예제)

```typescript
import { useState, useMemo } from 'react';

interface Student {
  id: number;
  name: string;
  grade: number;
  score: number;
}

const STUDENTS: Student[] = [
  { id: 1, name: "김철수", grade: 1, score: 85 },
  { id: 2, name: "이영희", grade: 2, score: 92 },
  { id: 3, name: "박민수", grade: 1, score: 78 },
  { id: 4, name: "정다은", grade: 3, score: 95 },
  { id: 5, name: "최준호", grade: 2, score: 88 },
];

type SortableKey = keyof Student;  // "id" | "name" | "grade" | "score"

function SortableTable() {
  const [sortKey, setSortKey] = useState<SortableKey>("id");
  const [sortOrder, setSortOrder] = useState<"asc" | "desc">("asc");

  // useMemo로 정렬 결과 캐시
  const sortedStudents = useMemo(() => {
    const sorted = [...STUDENTS].sort((a, b) => {
      const aVal = a[sortKey];
      const bVal = b[sortKey];

      if (typeof aVal === 'string' && typeof bVal === 'string') {
        return aVal.localeCompare(bVal);
      }
      return (aVal as number) - (bVal as number);
    });

    return sortOrder === "desc" ? sorted.reverse() : sorted;
  }, [sortKey, sortOrder]);  // sortKey 또는 sortOrder가 바뀔 때만 재정렬

  const handleHeaderClick = (key: SortableKey) => {
    if (sortKey === key) {
      setSortOrder(prev => prev === "asc" ? "desc" : "asc");
    } else {
      setSortKey(key);
      setSortOrder("asc");
    }
  };

  // 정렬 표시 아이콘
  const getSortIcon = (key: SortableKey) => {
    if (sortKey !== key) return "";
    return sortOrder === "asc" ? " ▲" : " ▼";
  };
```

```
  return (
    <table>
      <thead>
        <tr>
          <th onClick={() => handleHeaderClick("name")}>
            이름{getSortIcon("name")}
          </th>
          <th onClick={() => handleHeaderClick("grade")}>
            학년{getSortIcon("grade")}
          </th>
          <th onClick={() => handleHeaderClick("score")}>
            점수{getSortIcon("score")}
          </th>
        </tr>
      </thead>
      <tbody>
        {sortedStudents.map(student => (
          <tr key={student.id}>
            <td>{student.name}</td>
            <td>{student.grade}학년</td>
            <td>{student.score}점</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

---

key의 역할 심화

key가 바뀌면 컴포넌트가 완전히 새로 만들어짐

```
function App() {
  const [userId, setUserId] = useState(1);

  return (
    <div>
      <button onClick={() => setUserId(1)}>사용자 1</button>
      <button onClick={() => setUserId(2)}>사용자 2</button>

      {/* key가 바뀌면 UserProfile이 완전히 새로 생성됨 */}
      {/* 내부 state도 초기화됨! */}
      <UserProfile key={userId} userId={userId} />
    </div>
  );
}
```

핵심: `key` 가 같으면 React는 "같은 컴포넌트"로 인식하고, `key` 가 달라지면 "완전히 새로운 컴포넌트"로 인식합니다. 이를 활용해서 컴포넌트를 "리셋"할 수 있습니다.

Day 13 - useEffect와 커스텀 훅

useEffect란?

비유: "이 일이 끝나면 이것도 해줘"

식당에서 주문(렌더링)을 하고 나면, 추가 요청(사이드 이펙트)을 할 수 있습니다:

- "음식 나오면 사진 찍어줘" (렌더링 후 API 호출)
- "계산 끝나면 영수증 줘" (언마운트 시 정리)

`useEffect`는 **렌더링이 끝난 후 실행되는 코드**를 작성하는 훅입니다.

```
import { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  // useEffect 기본 구조
  useEffect(() => {
    // 이 코드는 렌더링이 끝난 후 실행됩니다
    console.log("렌더링 완료! 현재 seconds:", seconds);
  });
  // 의존성 배열이 없으면 → 매 렌더링마다 실행

  return <p>{seconds}초</p>;
}
```

의존성 배열에 따른 3가지 동작

```
// 1. 의존성 배열 없음: 매 렌더링마다 실행
useEffect(() => {
  console.log("매번 실행");
});

// 2. 빈 배열 []: 최초 한 번만 실행 (마운트 시)
useEffect(() => {
  console.log("처음 한 번만 실행");
}, []);

// 3. 값이 있는 배열: 해당 값이 바뀔 때만 실행
useEffect(() => {
  console.log("count가 바뀔 때마다 실행:", count);
}, [count]);
```

의존성 배열	실행 시점	실생활 비유
없음	매 렌더링마다	매일 아침 알람
`[]`	마운트 시 1번	입학식 (한 번)
`[count]`	count 변경 시	count가 바뀔 때마다 알림

사이드 이펙트 종류

API 호출

```typescript
interface User {
  id: number;
  name: string;
  email: string;
}

function UserProfile({ userId }: { userId: number }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // API 호출 (비동기)
    const fetchUser = async () => {
      try {
        setLoading(true);
        setError(null);

        const response = await fetch(`https://api.example.com/users/${userId}`);
        if (!response.ok) throw new Error("사용자를 찾을 수 없습니다");

        const data: User = await response.json();
        setUser(data);
      } catch (err) {
        setError(err instanceof Error ? err.message : "알 수 없는 오류");
      } finally {
        setLoading(false);
      }
    };

    fetchUser();
  }, [userId]);  // userId가 바뀌면 다시 호출

  if (loading) return <p>로딩 중...</p>;
  if (error) return <p>오류: {error}</p>;
  if (!user) return <p>사용자 없음</p>;

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
}
```

localStorage 저장/불러오기

```
function PersistentCounter() {
  // localStorage에서 초기값 불러오기
  const [count, setCount] = useState<number>(() => {
    const saved = localStorage.getItem("count");
    return saved ? JSON.parse(saved) : 0;
  });

  // count가 바뀔 때마다 localStorage에 저장
  useEffect(() => {
    localStorage.setItem("count", JSON.stringify(count));
  }, [count]);

  return (
    <div>
      <p>카운트: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>
      <p>페이지를 새로고침해도 값이 유지됩니다!</p>
    </div>
  );
}
```

클린업 함수: 메모리 누수 방지

비유: 방을 나올 때 불 끄기

호텔 방에 들어가면 불을 켜고(이펙트 실행), 나올 때 반드시 불을 꺼야(클린업) 합니다. 불을 안 끄면 전기가 낭비됩니다(메모리 누수).

```
useEffect(() => {
  // "방에 들어감" (이펙트 실행)
  console.log("이펙트 시작");

  // "방을 나올 때" (클린업 함수)
  return () => {
    console.log("이펙트 정리");
  };
}, []);
```

setInterval + clearInterval 예제

```
function StopWatch() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(false);

  useEffect(() => {
    if (!isRunning) return;  // 실행 중이 아니면 아무것도 안 함

    // 1초마다 seconds 증가
    const intervalId = setInterval(() => {
      setSeconds(prev => prev + 1);  // 함수형 업데이트 사용!
    }, 1000);

    // 클린업: 컴포넌트가 사라지거나 isRunning이 바뀌면 타이머 정리
    return () => {
      clearInterval(intervalId);  // 타이머 해제 (불 끄기!)
    };
  }, [isRunning]);

  return (
    <div>
      <p>{seconds}초</p>
      <button onClick={() => setIsRunning(!isRunning)}>
        {isRunning ? "정지" : "시작"}
      </button>
      <button onClick={() => { setIsRunning(false); setSeconds(0); }}>
        초기화
      </button>
    </div>
  );
}
```

addEventListener + removeEventListener 예제

```
function WindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    // 이벤트 핸들러 등록
    const handleResize = () => {
      setSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    };

    window.addEventListener('resize', handleResize);

    // 클린업: 이벤트 핸들러 제거 (안 하면 메모리 누수!)
    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []);  // 빈 배열: 마운트 시 등록, 언마운트 시 해제

  return (
    <p>창 크기: {size.width} x {size.height}</p>
  );
}
```

디바운스 패턴

비유: 엘리베이터 문

엘리베이터에 사람이 탈 때마다 문이 닫히려다가 다시 열립니다. 더 이상 사람이 안 들어오면 **일정 시간 후** 문이 닫힙니다. 디바운스도 마찬가지로, 사용자가 입력을 멈출 때까지 기다렸다가 마지막 값으로 한 번만 실행합니다.

```
function SearchWithDebounce() {
  const [input, setInput] = useState("");        // 실제 입력값 (즉시 반영)
  const [debouncedValue, setDebouncedValue] = useState("");  // 디바운스된 값

  // 디바운스: input이 바뀌고 300ms 동안 변화가 없으면 debouncedValue 업데이트
  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(input);
    }, 300);  // 300ms 대기

    // 클린업: input이 바뀌면 이전 타이머를 취소
    return () => clearTimeout(timer);
  }, [input]);

  // debouncedValue가 바뀔 때만 API 호출 (300ms에 한 번)
  useEffect(() => {
    if (debouncedValue === "") return;

    console.log("API 호출:", debouncedValue);
    // fetch(`/api/search?q=${debouncedValue}`)...
  }, [debouncedValue]);

  return (
    <div>
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        placeholder="검색어를 입력하세요..."
      />
      <p>검색 중: {debouncedValue}</p>
    </div>
  );
}
```

> **디바운스 없이 API 호출하면?** 사용자가 "React"를 입력하면 "R", "Re", "Rea", "Reac", "React" 총 5번 API가 호출됩니다. 디바운스를 사용하면 타이핑이 끝난 후 "React" 1번만 호출됩니다.

커스텀 훅

비유: 나만의 도구 만들기

`useState`, `useEffect`는 React가 제공하는 기본 도구입니다. 커스텀 훅은 이 기본 도구를 조합해서 **나만의 도구**를 만드는 것입니다.

규칙: `use` 접두사

커스텀 훅의 이름은 반드시 `use`로 시작해야 합니다. 이것은 React가 "이것은 훅이다"라고 인식하기 위한 규칙입니다.

useLocalStorage 커스텀 훅

```
// hooks/useLocalStorage.ts
import { useState, useEffect } from 'react';

function useLocalStorage<T>(key: string, initialValue: T) {
  // localStorage에서 값 읽기 (없으면 초기값 사용)
  const [value, setValue] = useState<T>(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved ? JSON.parse(saved) : initialValue;
    } catch {
      return initialValue;
    }
  });

  // value가 바뀔 때마다 localStorage에 저장
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
  // as const: 반환 타입을 [T, SetStateAction<T>] 튜플로 고정
}

export default useLocalStorage;
```

```
// 사용 예시: useState처럼 쓰되, 자동으로 localStorage에 저장됨!
function Settings() {
  const [name, setName] = useLocalStorage("userName", "");
  const [theme, setTheme] = useLocalStorage("theme", "light");

  return (
    <div>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <button onClick={() => setTheme(t => t === "light" ? "dark" : "light")}>
        테마: {theme}
      </button>
    </div>
  );
}
```

useDebounce 커스텀 훅

```typescript
// hooks/useDebounce.ts
import { useState, useEffect } from 'react';

function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

export default useDebounce;
```

```typescript
// 사용 예시: 복잡한 디바운스 로직이 한 줄로!
function Search() {
  const [input, setInput] = useState("");
  const debouncedSearch = useDebounce(input, 300);  // 300ms 디바운스

  useEffect(() => {
    if (debouncedSearch) {
      console.log("API 검색:", debouncedSearch);
    }
  }, [debouncedSearch]);

  return <input value={input} onChange={(e) => setInput(e.target.value)} />;
}
```

useFetch 커스텀 훅

```typescript
// hooks/useFetch.ts
import { useState, useEffect } from 'react';

interface UseFetchResult<T> {
  data: T | null;
  loading: boolean;
  error: string | null;
}

function useFetch<T>(url: string): UseFetchResult<T> {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        setLoading(true);
        setError(null);
        const response = await fetch(url);
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const json = await response.json();
        setData(json);
      } catch (err) {
        setError(err instanceof Error ? err.message : "오류 발생");
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, [url]);  // url이 바뀌면 다시 호출

  return { data, loading, error };
}

export default useFetch;
```

```
// 사용 예시: 복잡한 fetch 로직이 한 줄로!
function UserList() {
  const { data: users, loading, error } = useFetch<User[]>("/api/users");

  if (loading) return <p>로딩 중...</p>;
  if (error) return <p>오류: {error}</p>;
  if (!users) return null;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

---

useRef: DOM 직접 접근

비유: 이름표 붙이기

HTML 요소에 **이름표**(ref)를 붙여두면, 나중에 그 이름표로 해당 요소를 직접 찾을 수 있습니다.

```
import { useRef, useEffect } from 'react';

function AutoFocusInput() {
  // 1. ref 생성 (이름표 만들기)
  const inputRef = useRef<HTMLInputElement>(null);

  // 2. 마운트 시 자동 포커스
  useEffect(() => {
    // 3. ref.current로 실제 DOM 요소에 접근
    inputRef.current?.focus();  // 입력창에 자동으로 커서가 놓임
  }, []);

  return (
    <div>
      {/* ref 속성으로 이름표 붙이기 */}
      <input ref={inputRef} placeholder="자동으로 포커스됩니다" />
    </div>
  );
}
```

useRef의 또 다른 용도: 렌더링과 무관한 값 저장

```
function RenderCounter() {
  const [count, setCount] = useState(0);

  // useRef로 저장한 값은 바뀌어도 리렌더링을 일으키지 않음!
  const renderCount = useRef(0);

  useEffect(() => {
    renderCount.current += 1;  // 렌더링될 때마다 증가
  });

  return (
    <div>
      <p>카운트: {count}</p>
      <p>렌더링 횟수: {renderCount.current}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>
    </div>
  );
}
```

useState vs useRef:

- `useState` : 값이 바뀌면 화면이 다시 그려짐 (리렌더링)
- `useRef` : 값이 바뀌어도 화면은 안 바뀜 (리렌더링 없음)

Day 14 - React Router: 페이지 이동

SPA(Single Page Application)란?

비유: 한 건물 안에서 방 이동

기존 웹사이트(MPA): 매번 새 건물(HTML 파일)로 이동합니다.

- `/home` 페이지 → 서버에서 home.html 다운로드
- `/about` 페이지 → 서버에서 about.html 다운로드
- 매번 전체 페이지가 새로고침됨 (깜빡임)

SPA(React): 한 건물(index.html) 안에서 방(컴포넌트)만 바꿉니다.

- `/home` → App 안에서 HomePage 컴포넌트를 보여줌
- `/about` → App 안에서 AboutPage 컴포넌트를 보여줌
- 페이지 새로고침 없음! (부드러운 전환)

---

react-router-dom 설치와 설정

```
# react-router-dom 설치
npm install react-router-dom
```

기본 라우팅 설정

```
// main.tsx
import { BrowserRouter } from 'react-router-dom';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

```
// App.tsx
import { Routes, Route } from 'react-router-dom';
import HomePage from './pages/HomePage';
import AboutPage from './pages/AboutPage';
import TodoPage from './pages/TodoPage';
import NotFoundPage from './pages/NotFoundPage';

function App() {
  return (
    <div>
      {/* 네비게이션 (항상 보임) */}
      <Navigation />

      {/* 라우트 정의: URL에 따라 다른 컴포넌트를 보여줌 */}
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
        <Route path="/todos" element={<TodoPage />} />
        {/* *: 위의 어떤 경로에도 매칭되지 않으면 → 404 페이지 */}
        <Route path="*" element={<NotFoundPage />} />
      </Routes>
    </div>
  );
}
```

Link vs `<a>` 태그 차이

```
import { Link } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      {/* Link: 페이지 새로고침 없이 이동 (SPA 방식) */}
      <Link to="/">홈</Link>
      <Link to="/about">소개</Link>
      <Link to="/todos">할일</Link>

      {/* a 태그: 페이지 전체를 새로고침! (전통 방식) */}
      {/* <a href="/about">소개</a>  ← React 앱에서는 쓰지 마세요! */}
    </nav>
  );
}
```

	`<Link to="/about">`	`<a href="/about">`
페이지 새로고침	X (SPA)	O (전체 재로딩)
React 상태 유지	O	X (초기화됨)
사용 상황	앱 내부 이동	외부 사이트 링크

동적 라우트: URL에 변수 넣기

```
// App.tsx에 동적 라우트 추가
<Routes>
  <Route path="/todos" element={<TodoListPage />} />
  {/* :id 부분이 변수! /todos/1, /todos/2 등 어떤 값이든 매칭됨 */}
  <Route path="/todos/:id" element={<TodoDetailPage />} />
  <Route path="/users/:userId" element={<UserProfilePage />} />
</Routes>
```

useParams로 파라미터 가져오기

```
import { useParams } from 'react-router-dom';

function TodoDetailPage() {
  // URL의 :id 부분을 가져옴
  // /todos/42 → params.id = "42" (문자열!)
  const params = useParams<{ id: string }>();

  // 문자열을 숫자로 변환
  const todoId = Number(params.id);

  return (
    <div>
      <h1>할일 #{todoId} 상세 페이지</h1>
      {/* todoId를 사용해서 API 호출 등 */}
    </div>
  );
}
```

---

useNavigate: 코드로 페이지 이동

Link는 사용자가 클릭해서 이동하는 것이고, `useNavigate`는 **코드에서 프로그래밍적으로** 이동할 때 사용합니다.

```
import { useNavigate } from 'react-router-dom';

function LoginPage() {
  const navigate = useNavigate();

  const handleLogin = async () => {
    const success = await login(username, password);

    if (success) {
      // 로그인 성공 → 홈으로 이동
      navigate("/");
      // navigate("/", { replace: true });
      // replace: true → 뒤로가기로 로그인 페이지에 못 돌아옴
    }
  };

  return (
    <div>
      {/* 로그인 폼 */}
      <button onClick={handleLogin}>로그인</button>

      {/* 뒤로가기 */}
      <button onClick={() => navigate(-1)}>뒤로 가기</button>
    </div>
  );
}
```

인증 가드 (ProtectedRoute)

비유: 클럽 입구 보안요원

클럽에 들어가려면 보안요원(ProtectedRoute)이 신분증(로그인 상태)을 확인합니다. 신분증이 없으면 입구(로그인 페이지)로 돌려보냅니다.

```typescript
import { Navigate } from 'react-router-dom';

interface ProtectedRouteProps {
  isLoggedIn: boolean;            // 로그인 여부
  children: React.ReactNode;      // 보호할 페이지 컴포넌트
}

function ProtectedRoute({ isLoggedIn, children }: ProtectedRouteProps) {
  // 로그인 안 되어 있으면 → 로그인 페이지로 강제 이동
  if (!isLoggedIn) {
    return <Navigate to="/login" replace />;
    // replace: 뒤로가기로 보호된 페이지에 접근 못하게 함
  }

  // 로그인 되어 있으면 → 원래 페이지를 보여줌
  return <>{children}</>;
}
```

```typescript
// App.tsx에서 사용
function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  return (
    <Routes>
      {/* 누구나 접근 가능한 페이지 */}
      <Route path="/" element={<HomePage />} />
      <Route path="/login" element={<LoginPage onLogin={() => setIsLoggedIn(true)

      {/* 로그인해야 접근 가능한 페이지 */}
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute isLoggedIn={isLoggedIn}>
            <DashboardPage />
          </ProtectedRoute>
        }
      />
      <Route
        path="/profile"
        element={
          <ProtectedRoute isLoggedIn={isLoggedIn}>
            <ProfilePage />
          </ProtectedRoute>
        }
      />
    </Routes>
  );
}
```

RBAC (역할 기반 접근 제어)

사용자의 **역할(Role)**에 따라 접근 가능한 페이지를 다르게 설정합니다.

```
// 사용자 역할 타입
type Role = "user" | "admin" | "manager";

interface RoleGuardProps {
  userRole: Role;                    // 현재 사용자의 역할
  allowedRoles: Role[];              // 이 페이지에 접근 가능한 역할들
  children: React.ReactNode;
}

function RoleGuard({ userRole, allowedRoles, children }: RoleGuardProps) {
  // 현재 사용자의 역할이 허용 목록에 있는지 확인
  if (!allowedRoles.includes(userRole)) {
    // 권한 없음 → 접근 거부 페이지로
    return <Navigate to="/unauthorized" replace />;
  }

  return <>{children}</>;
}
```

```
// 사용 예시
<Routes>
  {/* 모든 로그인 사용자 접근 가능 */}
  <Route
    path="/dashboard"
    element={
      <ProtectedRoute isLoggedIn={isLoggedIn}>
        <DashboardPage />
      </ProtectedRoute>
    }
  />

  {/* admin만 접근 가능 */}
  <Route
    path="/admin"
    element={
      <ProtectedRoute isLoggedIn={isLoggedIn}>
        <RoleGuard userRole={currentUser.role} allowedRoles={["admin"]}>
          <AdminPage />
        </RoleGuard>
      </ProtectedRoute>
    }
  />

  {/* admin과 manager만 접근 가능 */}
  <Route
    path="/management"
    element={
      <ProtectedRoute isLoggedIn={isLoggedIn}>
        <RoleGuard userRole={currentUser.role} allowedRoles={["admin", "manager"]
          <ManagementPage />
        </RoleGuard>
      </ProtectedRoute>
    }
  />
</Routes>
```

자주 하는 실수

```
// 실수 1: BrowserRouter를 App 안에 넣음
function App() {
  return (
    <BrowserRouter>  {/* main.tsx에 한 번만 감싸세요! */}
      <Routes>...</Routes>
    </BrowserRouter>
  );
}

// 실수 2: Link를 BrowserRouter 바깥에서 사용
// Link, useNavigate 등은 반드시 BrowserRouter 안에서만 사용 가능!

// 실수 3: useParams의 반환값이 항상 string
const { id } = useParams();
// id는 string | undefined! 숫자가 필요하면 Number(id)로 변환!
```

Day 15 - Context API: 전역 상태 관리

Props Drilling 문제

비유: 전화 돌리기

회사에서 사장님(최상위 컴포넌트)의 메시지를 말단 직원(깊은 하위 컴포넌트)에게 전달한다고 합시다.

```
사장님 (App) → 전무 (Layout) → 부장 (Main) → 과장 (Section) → 대리 (List) → 사원 (Item)
```

중간에 있는 전무, 부장, 과장, 대리는 메시지 내용이 필요 없는데도 **그냥 전달만** 합니다. 이것이 바로 **Props Drilling** 문제입니다.

```reactjs
// Props Drilling 문제 예시
function App() {
  const [user, setUser] = useState({ name: "김철수", role: "admin" });
  const [theme, setTheme] = useState("light");

  return (
    // user와 theme을 계속 아래로 전달해야 함...
    <Layout user={user} theme={theme}>
      <Main user={user} theme={theme}>
        <Section user={user} theme={theme}>
          <Card user={user} theme={theme}>
            {/* 여기서 드디어 user와 theme을 사용! */}
            <UserAvatar user={user} />
          </Card>
        </Section>
      </Main>
    </Layout>
  );
}

// Layout, Main, Section, Card는 user와 theme을 직접 사용하지 않는데도
// 자식에게 전달하기 위해 props로 받아야 함 → 불필요한 코드!
```

왜 Context가 필요한가?

Context는 **사내 방송 시스템**과 같습니다. 사장님이 방송(Context)으로 메시지를 보내면, 듣고 싶은 사람(useContext)만 듣습니다. 중간 전달자가 필요 없습니다!

Context API 3단계

1단계: createContext (방송 채널 만들기)

```
import { createContext } from 'react';

// 테마 Context 생성
// "방송 채널"을 하나 만든다고 생각하세요
interface ThemeContextType {
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}

// createContext(기본값): 방송 채널 생성
const ThemeContext = createContext<ThemeContextType | null>(null);
```

2단계: Provider (방송하기)

```
import { useState } from 'react';

// Provider: "이 범위 안의 모든 컴포넌트에게 방송합니다"
function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };

  // value에 넣은 값이 "방송 내용"
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

3단계: useContext + 커스텀 훅 (방송 듣기)

```
import { useContext } from 'react';

// 커스텀 훅으로 만들면 사용이 편리하고 에러 처리도 가능
function useTheme() {
  const context = useContext(ThemeContext);

  // Provider 밖에서 사용하면 에러!
  if (!context) {
    throw new Error("useTheme은 ThemeProvider 안에서 사용해야 합니다!");
  }

  return context;
}
```

전체 연결: 테마 (다크모드) 구현

```tsx
// --- context/ThemeContext.tsx ---
import { createContext, useContext, useState } from 'react';

interface ThemeContextType {
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextType | null>(null);

// Provider 컴포넌트
export function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 커스텀 훅
export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme은 ThemeProvider 안에서 사용해야 합니다!");
  }
  return context;
}
```

```
// --- 어디서든 사용 (props 전달 필요 없음!) ---
function Header() {
  const { theme, toggleTheme } = useTheme();  // 방송을 듣는다!

  return (
    <header style={{
      backgroundColor: theme === 'dark' ? '#1a202c' : '#ffffff',
      color: theme === 'dark' ? '#e2e8f0' : '#1a202c',
    }}>
      <h1>나의 앱</h1>
      <button onClick={toggleTheme}>
        {theme === 'dark' ? '라이트 모드' : '다크 모드'}
      </button>
    </header>
  );
}

function Card({ title, content }: { title: string; content: string }) {
  const { theme } = useTheme();  // 여기서도 직접 접근 가능!

  return (
    <div style={{
      backgroundColor: theme === 'dark' ? '#2d3748' : '#f7fafc',
      color: theme === 'dark' ? '#e2e8f0' : '#1a202c',
      padding: '16px',
      borderRadius: '8px',
    }}>
      <h3>{title}</h3>
      <p>{content}</p>
    </div>
  );
}
```

TodoContext 구현

좀 더 복잡한 예제로, Todo 앱의 상태를 Context로 관리해봅시다.

```tsx
// --- context/TodoContext.tsx ---
import { createContext, useContext, useState } from 'react';

// 타입 정의
interface Todo {
  id: number;
  text: string;
  done: boolean;
}

interface TodoContextType {
  todos: Todo[];
  addTodo: (text: string) => void;
  toggleTodo: (id: number) => void;
  deleteTodo: (id: number) => void;
  editTodo: (id: number, text: string) => void;
}

// Context 생성
const TodoContext = createContext<TodoContextType | null>(null);

// Provider 컴포넌트: 모든 상태와 로직을 여기서 관리
export function TodoProvider({ children }: { children: React.ReactNode }) {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [nextId, setNextId] = useState(1);

  // 추가
  const addTodo = (text: string) => {
    setTodos(prev => [...prev, { id: nextId, text, done: false }]);
    setNextId(prev => prev + 1);
  };

  // 토글
  const toggleTodo = (id: number) => {
    setTodos(prev =>
      prev.map(todo =>
        todo.id === id ? { ...todo, done: !todo.done } : todo
      )
    );
  };

  // 삭제
  const deleteTodo = (id: number) => {
    setTodos(prev => prev.filter(todo => todo.id !== id));
  };

  // 수정
  const editTodo = (id: number, text: string) => {
    setTodos(prev =>
      prev.map(todo =>
        todo.id === id ? { ...todo, text } : todo
      )
```

```
    );
  };

  return (
    <TodoContext.Provider value={{ todos, addTodo, toggleTodo, deleteTodo, editTo
      {children}
    </TodoContext.Provider>
  );
}

// 커스텀 훅
export function useTodos() {
  const context = useContext(TodoContext);
  if (!context) {
    throw new Error("useTodos는 TodoProvider 안에서 사용해야 합니다!");
  }
  return context;
}
```

```
// --- components/TodoForm.tsx ---
// props 전달 없이 직접 Context에서 가져다 씀!
import { useState } from 'react';
import { useTodos } from '../context/TodoContext';

function TodoForm() {
  const [text, setText] = useState("");
  const { addTodo } = useTodos();  // Context에서 addTodo만 가져옴

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (text.trim() === "") return;
    addTodo(text);
    setText("");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="할일을 입력하세요"
      />
      <button type="submit">추가</button>
    </form>
  );
}
```

```
// --- components/TodoList.tsx ---
import { useTodos } from '../context/TodoContext';

function TodoList() {
  const { todos, toggleTodo, deleteTodo } = useTodos();

  if (todos.length === 0) {
    return <p>할일이 없습니다.</p>;
  }

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.done}
            onChange={() => toggleTodo(todo.id)}
          />
          <span style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>
            {todo.text}
          </span>
          <button onClick={() => deleteTodo(todo.id)}>삭제</button>
        </li>
      ))}
    </ul>
  );
}
```

```
// --- components/TodoStats.tsx ---
import { useTodos } from '../context/TodoContext';

function TodoStats() {
  const { todos } = useTodos();

  const total = todos.length;
  const done = todos.filter(t => t.done).length;
  const remaining = total - done;

  return (
    <div>
      <p>전체: {total} | 완료: {done} | 남은: {remaining}</p>
    </div>
  );
}
```

Provider 중첩 패턴 (main.tsx)

여러 Context를 사용할 때, Provider를 겹겹이 감쌉니다.

```
// main.tsx
import { BrowserRouter } from 'react-router-dom';
import { ThemeProvider } from './context/ThemeContext';
import { TodoProvider } from './context/TodoContext';
import { AuthProvider } from './context/AuthContext';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <BrowserRouter>
    <AuthProvider>
      <ThemeProvider>
        <TodoProvider>
          <App />
        </TodoProvider>
      </ThemeProvider>
    </AuthProvider>
  </BrowserRouter>
);
```

> **Provider의 순서:** 바깥쪽 Provider가 안쪽 Provider보다 먼저 초기화됩니다. 의존관계가 있으면 순서가 중요합니다. 예: AuthProvider가 바깥에 있어야 ThemeProvider에서 사용자별 테마를 설정할 수 있습니다.

언제 Context를 쓰고, 언제 Props를 쓰는가?

상황	Props	Context
부모→자식 1~2단계 전달	O	X
3단계 이상 깊이 전달	X	O
앱 전체에서 공유하는 데이터 (테마, 인증)	X	O
특정 컴포넌트만 사용하는 데이터	O	X
자주 바뀌는 데이터 (매 키 입력)	O	주의

> **Context 남용 주의:** Context의 value가 바뀌면, 그 Context를 사용하는 **모든 컴포넌트**가 다시 렌더링 됩니다. 자주 바뀌는 데이터(예: 입력값)를 Context에 넣으면 성능 문제가 발생할 수 있습니다.

Before vs After 비교

```
// Before: Props Drilling (중간 컴포넌트가 불필요한 props를 전달)
function App() {
  const [user, setUser] = useState(/*...*/);
  return <Layout user={user}><Main user={user}><Page user={user} /></Main></Layou
}

// After: Context (필요한 컴포넌트만 직접 접근)
function App() {
  return (
    <AuthProvider>
      <Layout><Main><Page /></Main></Layout>
    </AuthProvider>
  );
}

function Page() {
  const { user } = useAuth();  // 필요한 곳에서 직접 가져옴!
  return <p>{user.name}</p>;
}
```

자주 하는 실수

```
// 실수 1: Provider로 감싸지 않고 useContext 사용
function App() {
  return <TodoList />;  // TodoProvider로 감싸지 않음!
}
function TodoList() {
  const { todos } = useTodos();  // 에러! Provider가 없음!
}

// 실수 2: Context value에 매번 새 객체를 생성
function BadProvider({ children }) {
  const [count, setCount] = useState(0);

  // 렌더링될 때마다 새 객체가 생성되어 불필요한 리렌더링 발생!
  return (
    <MyContext.Provider value={{ count, setCount }}>
      {children}
    </MyContext.Provider>
  );
}

// 개선: useMemo로 value를 캐시
function GoodProvider({ children }) {
  const [count, setCount] = useState(0);

  const value = useMemo(() => ({ count, setCount }), [count]);

  return (
    <MyContext.Provider value={value}>
      {children}
    </MyContext.Provider>
  );
}
```

Phase 3 핵심 정리

Day	핵심 개념	한줄 요약
09	컴포넌트, JSX, Props	React는 레고 블록, Props는 택배
10	컴포넌트 분리, 콜백 Props	분업하고, 자식이 부모에게 전화
11	useState, CRUD	State는 기분, 불변성을 지켜라
12	필터/검색/정렬, useMemo	체이닝으로 조합, 비싼 계산은 캐시
13	useEffect, 커스텀 훅, useRef	렌더링 후 할 일, 나만의 도구 만들기
14	React Router	SPA는 한 건물에서 방 이동
15	Context API	사내 방송 시스템으로 Props Drilling 해결